

Project 1a: Testing WolfBite

Abigail Close

North Carolina State University
Raleigh, North Carolina, USA
aeclose2@ncsu.edu

Satwi Shah

North Carolina State University
Raleigh, North Carolina, USA
stshah2@ncsu.edu

Aditya Mahajan

North Carolina State University
Raleigh, North Carolina, USA
amahaja8@ncsu.edu

Supreme Constantine

North Carolina State University
Raleigh, North Carolina, USA
spconsta@ncsu.edu

Abstract

We reverse engineered the high-level design of WolfBite, a Flutter/Firebase WIC shopping assistant, using multiple large language models and direct repository evidence. We designed and executed tests against the resulting use cases, traced those tests to the design, and assessed the inherited suite only after completing the independent design pass. Our 86 team-designed tests executed independently of the inherited suite: 82 passed, three exposed unhandled or misleading user paths, and one exposed a test-scope mismatch. This report presents those findings without repairing the inherited application.

1 Product Choice (D1)

Product: WolfBite.

Original repository: https://github.com/SuyeshJadhav/CSC510_G19.

Team fork: https://github.com/closeabigaile/SEf26_project.

Production baseline commit: cc22b3c.

Curated-suite commit: 8461250.

We selected WolfBite because it is a user-facing Flutter application with several interconnected features, including authentication, barcode and receipt scanning, persistent basket state, WIC benefit limits, and nutritional calculations. These interactions provide meaningful state transitions, edge cases, and testing opportunities. The application could still be built and launched, although parts of its inherited test suite showed dependency rot and some features appeared incomplete or insufficiently defined. We did not evaluate and abandon another project before selecting WolfBite.

1.1 Build and Execution Baseline

The inherited web application built and displayed its sign-in page. During the initial test run, 22 tests passed, while five test files did not compile because fresh dependency resolution selected incompatible Firestore and fake-Firestore interfaces. We classified this as dependency/code rot and did not repair it. Git history on main contains separate Project 1a commits from all four authors and merge commits for each assigned use-case range.

2 Reverse-Engineered Use Cases (D2)

The final use cases reconcile Codex, Gemini, GPT-5.6-terra, and a limited local Llama review. Repository evidence settled disagreements: barcode and manual UPC entry were combined as one identification goal; monthly reset remains an extension of session restoration; direct lookup is not described as guaranteed WIC verification; receipt import is distinct from checkout; and QR preparation is distinct from finishing the local shopping session.

2.1 UC1: Create account

Primary actor: New shopper

Stakeholders and interests: Shopper: obtain a usable account and retain profile information. Service owner: maintain valid, attributable accounts.

Preconditions: Shopper is signed out and account services are available.

Trigger: Shopper chooses to create an account.

Main success scenario: 1. Shopper provides the required account and profile information. 2. System validates the information. 3. System creates the shopper's account. 4. System saves the shopper's profile. 5. System ends the initial session and makes sign-in available.

Extensions: 2a. Required information is blank or invalid: system identifies the invalid entry and does not submit. 3a. Account creation is rejected: system reports the failure and keeps account creation available. 4a. Account creation succeeds but profile saving fails: the account and profile are left incomplete, with no reliable recovery in this flow.

Postconditions: The account and profile exist, and the shopper can sign in.

2.2 UC2: Sign in

Primary actor: Returning shopper

Stakeholders and interests: Shopper: gain secure access to saved shopping information. Service owner: reject invalid credentials and protect account data.

Preconditions: Shopper is signed out and has an account.

Trigger: Shopper chooses to sign in.

Main success scenario: 1. Shopper provides account credentials. 2. System validates the credentials. 3. System authenticates the shopper. 4. System makes protected shopping features available.

Extensions: 2a. Email format is invalid: system reports the invalid entry without authenticating. 2b. Password is shorter than six characters: system reports the minimum length. 3a. Authentication is rejected or unavailable: system reports the failure and leaves the shopper signed out. 3b. A sign-in request is active: system prevents another submission until it finishes.

Postconditions: Shopper is authenticated and can access protected shopping features.

2.3 UC3: Sign out

Primary actor: Authenticated shopper

Stakeholders and interests: Shopper: prevent later device users from accessing the account. Service owner: remove account-specific information from the active session.

Preconditions: Shopper is signed in.

Trigger: Shopper chooses to sign out.

Main success scenario: 1. Shopper requests sign-out. 2. System ends the authenticated session. 3. System removes account-specific shopping information from the current session. 4. System makes sign-in available.

Extensions: 2a. Sign-out fails: shopper remains signed in and receives no explicit explanation. 4a. Signed-out state is detected before the normal transition finishes: system still returns the shopper to sign-in.

Postconditions: No shopper is authenticated locally, and protected features require sign-in.

2.4 UC4: Resume shopping session

Primary actor: Returning shopper

Stakeholders and interests: Shopper: recover the correct basket and benefit usage. Service owner: keep each shopper's information isolated.

Preconditions: Shopper has authenticated and has an account identifier.

Trigger: System detects the signed-in shopper.

Main success scenario: 1. System identifies the shopper. 2. System retrieves saved shopping information. 3. System restores the basket, quantities, product information, and benefit usage. 4. System confirms the activity belongs to the current benefit month. 5. System makes the restored session available.

Extensions: 2a. No saved information exists: system begins an empty session. 2b. Retrieval fails: loading finishes without user-facing recovery. 3a. Saved basket or benefit information is missing: available information is restored and empty values replace the rest. 3b. Saved nutrition is absent: zero-valued nutrition is substituted. 4a. Activity is from an earlier month or year: basket and used benefits reset. 4b. Saving the reset fails: the visible reset may not be preserved.

Postconditions: The shopper's current-month saved session, or a new empty session, is available.

2.5 UC5: Identify product

Primary actor: Shopper

Stakeholders and interests: Shopper: identify a product without relying on unsupported claims. Product-data owner: receive the intended code.

Preconditions: Shopper is signed in and product lookup is available.

Trigger: Shopper provides a product code by barcode capture or manual entry.

Main success scenario: 1. Shopper provides a product code. 2. System searches for a matching product record. 3. System identifies the product. 4. System presents its category, code, and nutrition summary. 5. System makes healthier alternatives available.

Extensions: 1a. Barcode capture is cancelled or unreadable: system remains ready. 1b. Manual entry is empty: no lookup occurs and no explanation is given. 2a. Another lookup is active: the new request is ignored. 2b. No record matches: system reports not found and clears the current result. 2c. Lookup fails: system reports the failure. 3a. Record is ineligible or lacks eligibility: system still presents it because direct lookup does not enforce eligibility.

Postconditions: A matching product is available for review, and the basket is unchanged.

2.6 UC6: Review product nutrition

Primary actor: Health-conscious shopper

Stakeholders and interests: Shopper: understand the product's nutritional qualities. Nutrition-data owner: have available values interpreted consistently.

Preconditions: A product has been identified.

Trigger: Shopper reviews the identified product.

Main success scenario: 1. System evaluates available nutrition information. 2. System presents up to three applicable nutritional qualities. 3. Shopper reviews the summary.

Extensions: 1a. Nutrient is missing or nonnumeric: zero is substituted. 1b. Value equals a threshold: the applicable quality is included. 1c. Missing values become zero: misleading low or heart-healthy qualities may appear. 2a. No rule matches: no quality label appears. 2b. More than three rules match: only the first three appear.

Postconditions: Shopper has reviewed the nutrition summary; product and basket are unchanged.

2.7 UC7: Compare healthier alternatives

Primary actor: Health-conscious shopper

Stakeholders and interests: Shopper: find a healthier eligible option in the same category. Product-data owner: have candidates compared consistently.

Preconditions: An identified product has a nonempty category.

Trigger: Shopper requests healthier alternatives.

Main success scenario: 1. System compares eligible same-category products. 2. System identifies products with a better nutrition assessment. 3. System ranks them. 4. System presents up to five alternatives. 5. Shopper reviews the comparison.

Extensions: 1a. Category is blank: system does not search. 1b. Search fails: system reports failure. 2a. Candidate nutrition is missing: zero values may make it appear healthier. 2b. Candidate is the original product: system excludes it. 2c. No candidate is better: system reports no healthier alternative. 4a. More than fifty candidates exist: alternatives come only from the first fifty retrieved.

Postconditions: Shopper has reviewed up to five alternatives; basket is unchanged.

2.8 UC8: Add product to basket

Primary actor: WIC shopper

Stakeholders and interests: Shopper: add an identified product. Benefit program: keep covered usage within the category allowance.

Preconditions: Shopper is signed in and a product has been identified.

Trigger: Shopper chooses to add the identified product.

Main success scenario: 1. System confirms the product is within allowance. 2. System adds one unit. 3. System increases used benefits. 4. System saves and confirms the addition.

Extensions: 1a. Allowance is exhausted: covered addition is disabled and a warning appears. 2a. Product already exists: quantity-increase flow is used. 2b. Shopper is no longer authenticated: addition is rejected. 4a. Saving fails: the session may show an addition that was not preserved.

Postconditions: One covered unit is represented in the basket and category usage increased by one.

2.9 UC9: Choose healthier alternative

Primary actor: Health-conscious shopper

Stakeholders and interests: Shopper: choose a healthier product without another lookup. Benefit program: apply category allowances.

Preconditions: Healthier alternatives are available and shopper is signed in.

Trigger: Shopper selects a healthier alternative.

Main success scenario: 1. Shopper chooses an alternative. 2. System confirms allowance. 3. System adds the alternative. 4. System updates benefit usage. 5. System saves and confirms it.

Extensions: 2a. Allowance is exhausted: system may reject the addition but still report success. 3a. Alternative already exists: its quantity increases. 3b. Shopper is no longer authenticated: addition is rejected but may be reported successful. 5a. Saving fails: visible and restored baskets may differ.

Postconditions: On actual success, the selected alternative and benefit usage are saved in the basket.

2.10 UC10: Scan receipt

Primary actor: WIC shopper

Stakeholders and interests: Shopper: identify receipt products with less manual entry. Receipt-reading provider: receive a supported image. Product-data owner: validate extracted codes.

Preconditions: Shopper is signed in and required image, network, receipt-reading, and lookup services are available.

Trigger: Shopper chooses to scan a receipt.

Main success scenario: 1. Shopper supplies a receipt image. 2. System reads it. 3. System identifies contiguous 12–14 digit candidates. 4. System checks for matching records. 5. System presents unique matches and a summary.

Extensions: 1a. Selection is cancelled: attempt ends. 2a. Reading fails: failure is reported. 3a. No candidate exists: system reports none. 3b. Code contains spaces or hyphens: it is missed. 4a. Candidate has 13/14 digits: each 12-digit window is checked. 4b. None match: zero matches are reported. 5a. Code repeats: product appears once although recognition count includes repeats.

Postconditions: Zero or more unique matching products are available; basket is unchanged.

2.11 UC11: Add receipt products

Primary actor: WIC shopper

Stakeholders and interests: Shopper: transfer recognized products into the basket. Benefit program: apply category allowances.

Preconditions: Receipt scanning found a matching product and shopper is signed in.

Trigger: Shopper chooses to add recognized products.

Main success scenario: 1. System evaluates each product against allowances. 2. System adds accepted products. 3. System updates benefit usage. 4. System reports additions. 5. System presents the basket.

Extensions: 1a. Nutrition is absent: zero values are substituted. 2a. Product already exists: quantity increases but reported count omits it. 2b. Category is full: product is rejected rather than made shopper-paid. 5a. Saving fails: locally updated contents may not be preserved.

Postconditions: Every accepted product is represented in the basket and its category usage is updated.

2.12 UC12: Review basket

Primary actor: Shopper

Stakeholders and interests: Shopper: verify products and quantities. Benefit program: distinguish covered and shopper-paid quantities.

Preconditions: Shopper is signed in.

Trigger: Shopper requests the basket.

Main success scenario: 1. System presents each product, quantity, category, and nutrition summary. 2. System presents total quantity. 3. Shopper reviews the basket.

Extensions: 1a. Basket is empty: system explains and offers product identification. 1b. Nutrition is absent: zero values appear. 1c. Product has covered and paid quantities: they appear separately.

Postconditions: Shopper has reviewed the basket; quantities and benefit usage are unchanged.

2.13 UC13: Review basket-product nutrition

Primary actor: Health-conscious shopper

Stakeholders and interests: Shopper: inspect detailed nutrition facts. Nutrition-data owner: preserve available values.

Preconditions: Basket contains the product.

Trigger: Shopper requests detailed nutrition for a basket product.

Main success scenario: 1. Shopper selects a product. 2. System presents calories, fats, sodium, sugars, and protein. 3. Shopper reviews them. 4. Shopper returns to the basket summary.

Extensions: 2a. Nutrition is absent: zero defaults appear. 2b. One value is absent: system displays null instead of unknown. 4a. Shopper does not return: details remain visible while the view is retained.

Postconditions: Shopper has inspected nutrition; basket is unchanged.

2.14 UC14: Increase product quantity

Primary actor: WIC shopper

Stakeholders and interests: Shopper: add another unit. Benefit program: keep covered quantity within allowance.

Preconditions: Basket contains the selected covered product.

Trigger: Shopper requests another unit.

Main success scenario: 1. System confirms another covered unit is available. 2. System increases quantity. 3. System increases used benefits. 4. System saves and presents the basket.

Extensions: 1a. Allowance is exhausted: continue with UC15. 2a. Matching product is absent: no quantity changes although an update is processed. 4a. Saving fails: displayed and restored quantities may differ.

Postconditions: Covered quantity and used benefits increased by one.

2.15 UC15: Add shopper-paid quantity

Primary actor: Shopper

Stakeholders and interests: Shopper: obtain another unit after coverage is exhausted. Benefit program: exclude it from covered usage. Retailer: distinguish shopper-paid quantity.

Preconditions: Basket contains the covered product and its allowance is exhausted.

Trigger: Shopper requests another unit.

Main success scenario: 1. System identifies the unit as shopper-paid. 2. System adds it. 3. System leaves covered usage unchanged. 4. System saves and presents both quantities.

Extensions: 1a. Shopper adds a different newly identified product at the cap: addition is disabled rather than made paid. 2a. Paid quantity exists: it increases. 2b. Original covered product is absent: nothing is added. 4a. Benefits are reviewed: paid appears as unlimited although not a benefit. 4b. Saving fails: displayed and restored quantities may differ.

Postconditions: One shopper-paid unit is added and covered usage is unchanged.

2.16 UC16: Decrease product quantity

Primary actor: Shopper

Stakeholders and interests: Shopper: correct an over-selection. Benefit program: release usage only for a covered unit.

Preconditions: Basket contains the product.

Trigger: Shopper requests removal of one unit.

Main success scenario: 1. System identifies the applicable quantity. 2. System decreases it. 3. System adjusts usage for a covered unit. 4. System removes zero-quantity selection. 5. System saves and presents the basket.

Extensions: 1a. Covered and paid quantities exist: paid decreases first. 1b. No paid quantity exists: covered decreases. 1c. No match exists: no change. 3a. Usage is zero: it remains zero. 5a. Saving fails: displayed and restored quantities may differ.

Postconditions: One matching unit is removed and usage is not negative.

2.17 UC17: Clear basket

Primary actor: Shopper

Stakeholders and interests: Shopper: abandon all selections. Benefit program: release provisional usage.

Preconditions: Basket is nonempty.

Trigger: Shopper chooses to clear the basket.

Main success scenario: 1. System requests confirmation. 2. Shopper confirms. 3. System removes all selections. 4. System reverses provisional usage without making it negative. 5. System saves and presents the empty basket.

Extensions: 2a. Shopper cancels: nothing changes. 4a. Category is absent from balances: product is removed without a category change. 4b. Usage is less than removed quantity: result stays zero. 5a. Saving fails: current basket is empty but saved products may remain.

Postconditions: Basket is empty and its provisional usage is reversed.

2.18 UC18: Review benefit balances

Primary actor: WIC shopper

Stakeholders and interests: Shopper: know used, allowed, and remaining quantities. Benefit program: communicate limits accurately.

Preconditions: Shopper is signed in and information has loaded.

Trigger: Shopper requests benefit information.

Main success scenario: 1. System presents used and allowed quantities. 2. System distinguishes limited and unlimited categories. 3. System presents remaining capacity. 4. Shopper reviews balances.

Extensions: 1a. Information is loading: loading state appears. 1b. No categories exist: absence is reported. 2a. Paid exists: it appears unlimited although not a benefit. 3a. Usage reaches allowance: full progress appears. 3b. Allowance is zero: progress may be meaningless.

Postconditions: Shopper has reviewed benefits; basket and balances are unchanged.

2.19 UC19: Prepare checkout handoff

Primary actor: Shopper

Stakeholders and interests: Shopper: present selected products for checkout. Cashier: receive a scannable basket representation.

Preconditions: Shopper is signed in and basket is nonempty.

Trigger: Shopper declares the basket ready.

Main success scenario: 1. Shopper requests a handoff. 2. System prepares a scannable basket representation. 3. System presents it. 4. Shopper presents it to the cashier.

Extensions: 2a. Representation is unusable: no fallback exists. 4a. Cashier cannot consume it: no acknowledgement or interoperability check exists. 4b. Shopper leaves before finishing: basket and usage remain unchanged.

Postconditions: Basket remains available and its handoff is presented; external acceptance is not guaranteed.

2.20 UC20: Finish shopping session

Primary actor: Shopper

Stakeholders and interests: Shopper: finish checkout. Benefit program: retain consumed usage. Service owner: save the cleared basket consistently.

Preconditions: Shopper is signed in and checkout handoff is displayed.

Trigger: Shopper marks the transaction finished.

Main success scenario: 1. Shopper marks checkout finished. 2. System clears the basket. 3. System retains benefit usage. 4. System saves the cleared basket. 5. System confirms and returns to product identification.

Extensions: 1a. Shopper finishes without cashier confirmation: system proceeds. 4a. Saving fails: current basket remains cleared, no rollback occurs, and completion is not confirmed.

Postconditions: Basket is empty in saved state and benefit usage represents purchased quantities.

3 Designed Tests and Results (D3)

The repository contains the four assigned Project 1a test files, the 11-test gap file, its per-test expected-versus-actual results, Aditya's 23 per-test results, complete expanded output for the current team suite, and their per-test result narratives. The six appendix tables, Tables 15– 20, reproduce the required rationale, expected result, and actual result for every active test; the smaller tables below highlight representative findings. The four independently designed files initially executed 76 tests: 72 passed and four failed. Curation removed one overlapping team test, added 11 tests for transition and persistence gaps, and retained the team-authored UC19 test so that every use case has team-designed evidence. The current Project 1a suite therefore contains 86 tests. The prior project's tests are excluded from D3 and assessed separately in D4, as required by the assignment.

Table 1: Executed-suite summary

Scope	Executed	Passed	Failed
Initial team files	76	72	4
Current Project 1a suite	86	82	4

The recorded final command selected `test/project1a` and `test/active/project1a_coverage_gaps_test.dart`, then used `--concurrency=1--coverage--reporterexpanded` on Flutter 3.47.2. It exited with status 1. Failures in UC1, UC3, and UC9 exposed unhandled or misleading user paths. UC7-T6 also failed, but it directly called the service with a blank category while the production screen skips that call; it is a test-scope mismatch, not evidence of a user-visible UC7 failure. All 11 gap tests and the restored UC19 test passed. No inherited tests or SDK-renderer failures are included in these D3 totals.

Table 2: Representative test results (1 of 2)

Test	Why / expected / observed
<code>test_uc01_valid_registration_creates_account_and_profile</code>	Why: UC1 crosses Authentication, Firestore, sign-out, and navigation. Expected: create account/profile and open login. PASS: exact credentials/UID were used, sign-out called, and /login opened.
<code>test_uc01_profile_save_failure_does_not_complete_registration_flow</code>	Why: UC1 extension 4a separates account creation from profile persistence. Expected: report failure; do not claim completion. FAIL: the exception escaped with no message; sign-out/navigation did not occur.
<code>test_uc03_sign_out_failure_leaves_shopper_signed_in</code>	Why: UC3 extension 2a requires a failed sign-out to remain failed. Expected: preserve state and handle the error. FAIL: login navigation was withheld, but the exception escaped unexplained.
<code>UC7-T6blankcategory->nosearchresults</code>	Why: probe the blank-category boundary. Expected: no search or alternatives. TEST-SCOPE FAIL: the direct service call returned a product; the production screen does not make this call when category is blank.

Table 3: Representative test results (2 of 2)

Test	Why / expected / observed
<code>UC9-T2rejectedalternative->mustnotreportsuccess</code>	Why: UC9 must honor <code>AppState.addItem's</code> rejection. Expected: suppress success. FAIL: UI displayed <i>Added healthier item: Better cereal</i> .
<code>test_uc11_duplicate_recognized_upc_increments_but_addition_count_is_undercounted</code>	Why: UC11 extension 2a predicts quantity/report divergence. Expected: quantity two; second return false. PASS: both conditions occurred, confirming underreporting.
<code>test_uc16_paid_quantity_is_decreased_before_covered_quantity</code>	Why: UC16 extension 1a protects covered usage. Expected: paid unit removed first. PASS: paid was removed; covered quantity/usage stayed unchanged.

3.1 Raw Output Samples

The committed team-suite log preserves the complete expanded execution of all 86 current Project 1a tests. Short excerpts show both conforming and nonconforming executions:

The test description was:
`test_uc03_sign_out_failure_leaves_shopper_signed_in`
 00:02 +23 -2: Some tests failed.
 00:02 +15: All tests passed!

The failure excerpt is from UC1–UC5; the passing counter is from the original UC11–UC15 run. The current expanded log ends at 82 passes and four named failures, providing the raw-output evidence for the D3 totals above.

4 Traceability and Inherited Tests (D4)

The complete test-level traceability sources are available in the repository; the 11 post-curation tests also have a dedicated exact-name traceability table. Tables 4–7 account for all 86 current team-designed tests by stable test prefix or ID and include the 11 post-curation gap tests. Grouping repeated tests by use case keeps the ACM report readable without substituting test names for evidence.

Table 4: Team test-to-use-case traceability, UC1–UC5

Active tests	UC	What the assertions prove
test_uc01_* (5)	UC1	Valid creation, local validation, authentication rejection, and the unhandled profile-save failure.
test_uc02_* (5)	UC2	Valid navigation, invalid email, six-character boundary, credential rejection, and single pending submission.
test_uc03_* (5)	UC3	Sign-out invocation/navigation, local-state cleanup, already-signed-out routing, and the unhandled failure.
test_uc04_* (5)	UC4	Current, absent, partial, missing-nutrition, and previous-period saved sessions.
test_uc05_* (5)	UC5	Matching, empty, unknown, failed, and concurrent UPC lookups without basket mutation.

Table 5: Team test-to-use-case traceability, UC6–UC10

Active tests	UC	What the assertions prove
UC6–T1–UC6–T5 (5)	UC6	Normalization, zero defaults, inclusive thresholds, no-match behavior, and the three-badge limit.
UC7–T1–UC7–T6 (6)	UC7	Filtering, ranking, five-result limit, zero-default risk, no-better result, and the direct-service blank-category mismatch.
UC8–T1–UC8–T5 plus persistence (6)	UC8	Accepted/capped/duplicate/signed-out/category-normalized additions and successful persistence.
UC9–T1–UC9–T2 (2)	UC9	Exact alternative submission and the false success message after state rejection.
UC10–T1–UC10–T5 (5)	UC10	Source and picker cancellation, OCR failure, no-candidate input, and spaced/hyphenated code rejection.

Table 6: Team test-to-use-case traceability, UC11–UC15

Active tests	UC	What the assertions prove
test_uc11_* (3)	UC11	Multiple receipt products, duplicate-UPC quantity/reporting behavior, and missing-nutrition defaults.
test_uc12_* (3)	UC12	Product/category/total display, empty state, and separate covered/paid lines.
test_uc13_* (3)	UC13	Nutrition expansion, zero defaults, and return to the collapsed summary.
test_uc14_* plus three gap tests (5)	UC14	UI delegation below/at cap, real under-cap mutation, missing-product no-op, and persistence.
test_uc15_* plus missing-original gap (4)	UC15	Paid-line creation/increment, uncapped PAID state, and no paid line without the original product.

Table 7: Team test-to-use-case traceability, UC16–UC20

Active tests	UC	What the assertions prove
test_uc16_* plus persistence (6)	UC16	Covered decrease, removal at zero, paid-first order, missing-product no-op, nonnegative usage, and persistence.
test_uc17_* plus three gap tests (7)	UC17	Single/multi-category clearing, missing category, nonnegative usage, persistence, and confirm/cancel UI paths.
test_uc18_* (2)	UC18	PAID's unlimited presentation and full progress at allowance.
test_uc19_qr_code_contains_current_ basket_data plus inherited QR test	UC19	Both active tests check that a QR widget is rendered for a nonempty basket; payload content and external scan/cashier acceptance are not proved.
test_uc20_* plus two gap tests (3)	UC20	Basket clearing with retained usage, persisted cleared state, and failure leaving local state cleared.

4.1 Coverage Gaps

All 86 active team-designed tests map to at least one final use case; there are no orphan tests. All 20 use cases have an active team-designed test after the team-written UC19 QR-rendering test was

restored. At the main-outcome level, 18 of 20 use cases are covered. UC10 has team tests for error and cancellation extensions but lacks an offline happy-path OCR-to-matched-product test because the screen hard-codes `ApIService`, `ImagePicker`, and the live OCR

request. UC19 has a team QR-rendering test but lacks an assertion over the encoded payload or external cashier interoperability.

Table 8: Project 1a team-suite coverage audit

Criterion	Covered	Total	Percent
Executable lines	891	1,150	77.5%
Main outcomes	18	20	90.0%
Requirement paths	69	106	65.1%
Decision outcomes	55	84	65.5%
Exception paths	7	19	36.8%
Persistence outcomes	10	23	43.5%

LCOV contained no branch records, so a numeric source-branch percentage would be fabricated. Uncovered Project 1a paths include exception and persistence behavior, UC10’s 13/14-digit window plus unmatched and duplicate OCR candidates, UC19 payload validation, UC4 retrieval/reset-save failure, UC7 backend failure, and UC9 duplicate/persistence behavior. The suite also contains 18 meaningful boundary checks; no percentage is assigned because the possible-boundary denominator is not finite.

4.2 Assessment of the Inherited Tests

The inherited suite was inspected only after the independent use-case and test design pass. Nineteen overlapping inherited declarations that had been removed during suite curation were restored exactly to their original six files. They remain excluded from the explicitly selected D3 command but available unchanged for this D4 assessment. In the as-is baseline, 22 tests in signup, login, and balances passed, but five files (state, basket, scan, QR checkout, and APL service) did not compile. Fresh dependency resolution paired Firestore 6.8.0 with fake-Firestore 4.1.1, whose WriteBatch.update interfaces are incompatible. Source assertions in those files describe intent, not executable baseline evidence; the dependency rot was reported and not repaired.

Table 9: Inherited tests, UC1–UC5

UC	Verdict	Blind spots
UC1	Partial, executable	No profile-save failure after account creation or incomplete-profile recovery.
UC2	Largely covered	No direct proof that repeated pending submission makes only one request or that every protected route stays closed.
UC3	Partial	No sign-out exception handling, account-specific state cleanup, already-signed-out path, or cross-user isolation.
UC4	Weak; baseline blocked	One intended restore test; no missing/partial session, zero-default nutrition, month/year reset, retrieval failure, or reset-save failure.
UC5	Strong source intent; baseline blocked	No empty-input/stale-result sequence, cancelled barcode, unchanged-basket proof, or false/missing eligibility case.

Table 10: Inherited tests, UC6–UC10

UC	Verdict	Blind spots
UC6	Partial source intent	Scan rendering touches nutrition, but thresholds, missing/nonnumeric values, no-match, and the three-badge limit were not established.
UC7	Partial source intent	Same-category eligible substitutes were intended; blank category, ranking, original exclusion, limit, missing nutrition, and backend failure were blind.
UC8	Partial source intent	Basic Add/cap UI existed; signed-out rejection, category normalization, return-value meaning, and persistence outcomes were blind.
UC9	Weak source intent	The alternatives sheet was rendered, but state rejection, false success reporting, duplicate quantity, and persistence were untested.
UC10	None	No inherited receipt-scanner tests; OCR cancellation, parsing windows, deduplication, and service failures were blind.

Table 11: Inherited tests, UC11–UC15

UC	Verdict	Blind spots
UC11	None	No receipt-to-basket tests, including duplicate-count underreporting, cap rejection, or missing nutrition.
UC12	Partial; baseline blocked	Basic basket rendering/control intent, but no dependable empty-state or covered/paid separation evidence.
UC13	Weak; baseline blocked	No direct missing/partial nutrition or expand-collapse requirement coverage.
UC14	Partial; baseline blocked	Basic increment intent, but no missing-product, at-cap transition, or persistence evidence.
UC15	Weak	No production-state proof of paid-line creation/increment, missing original, covered-usage invariance, or the unlimited-display quirk.

Table 12: Inherited tests, UC16–UC20

UC	Verdict	Blind spots
UC16	Partial; baseline blocked	Basic decrement/removal intent, but no paid-first order, missing product, nonnegative usage, or persistence.
UC17	None	Confirmation/cancellation, multi-category reversal, missing category, nonnegative clamp, and persistence were blind.
UC18	Useful, executable	Loading, empty, limited, and generic unlimited displays existed; PAID-specific semantics and exact-cap progress were absent.
UC19	Partial; baseline blocked	A QR widget was rendered from basket JSON, but payload content, unusable QR, and cashier interoperability were not validated.
UC20	Misleading test double	The inherited checkout double reset usage unlike production; it cannot prove benefit retention or persistence-failure behavior.

Overall, inherited coverage was strongest in authentication UI and basic balance presentation, and weakest in receipt processing, persistence/error paths, month restoration, paid-overflow semantics, and checkout correctness. Three user-facing Project 1a failures occupy those blind spots: profile persistence, sign-out failure, and false success after a rejected alternative. The fourth Project 1a failure, UC7-T6, bypasses the screen’s blank-category guard and therefore identifies a test-to-use-case mismatch rather than a WolfBite user-flow failure.

5 Cross-Model Comparison and Prompt Notes (D5)

Complete raw model transcripts and model-specific audit notes are stored in the repository.

Table 13: Prompt-by-model verdicts, K1–K2

Prompt	Model	Verdict
K1	Codex Sol	Defensible 20; reset is an extension and tracking is absent.
K1	Terra	Strong after narrowing eligibility, capacity, and transaction wording.
K1	Gemini	Useful inventory; overstated eligibility, swap, signup outcome, and checkout.
K1	Local	Fabricated citations and promoted helpers to goals.
K2	Codex Sol	Found paid overflow, receipt/checkout separation, and unawaited saves.
K2	Terra	Best evidence/inference labels; qualified persistence and pre-checkout invariant.
K2	Gemini	Useful trace; contradicted itself at the cap and overstated reset timing.
K2	Local	Shallow; honestly used N/A when context was absent.

Table 14: Prompt-by-model verdicts, K3–K4

Prompt	Model	Verdict
K3	Codex Sol	Specific cap, rollover, OCR, nutrition, and paid-first boundaries.
K3	Terra	Strong source of nontrivial extensions and tests.
K3	Gemini	Valuable selectively; many entries were redesign proposals.
K3	Local	Generic boundaries with no valid evidence citations.
K4	Codex Sol	Found state README/CI and weaker eligibility/checkout claims.
K4	Terra	Strong partial/absent claim audit; verdicts retained.
K4	Gemini	Tracking correct; receipt, eligibility, checkout, and platform claims needed narrowing.
K4	Local	Inferred implementation from dependency names.

Repository evidence settled the disagreements: direct UPC lookup does not filter eligibility; signup signs out; routes contain no tracking; several state saves are unawaited; checkout retains usage; receipt Add All does not call checkout; production thresholds, OCR windows, rollover, and paid-first decrement define the real boundaries; and CI targets missing Project2.

5.1 Errors Caught

- **Codex Sol:** initially allowed the domain label *approved* to imply guaranteed WIC eligibility. Inspection of `ApIService.findByUpc` showed that any matching document is returned; only substitute lookup filters *eligible*. Code/route inventory also disproved README order tracking and separated receipt recognition, Add All, QR handoff, and local session completion.
- **GPT-5.6-terra:** promoted category-cap checking and *complete transaction* to actor goals, stated persistence too strongly, and applied a basket/usage equality invariant after checkout. The state code showed that capacity is subordinate to add/review flows, saves are often unawaited, and post-checkout usage intentionally remains while the basket is empty.
- **Gemini:** claimed signup enters the app, an alternative is atomically swapped, direct lookup verifies WIC eligibility, and checkout is permanently committed. The signup, scan, APL, QR, and state paths instead show sign-out-to-login, additive selection, unfiltered direct lookup, and no cashier acknowledgement. Its K4 receipt verdict also contradicted its own explanation and was corrected to partial/misleading.
- **Local Llama 3 8B:** fabricated file-line citations, turned private persistence/notification helpers into user goals, and

inferred QR implementation from the dependency list. Manual comparison against the cited files found no grounding, so none of its unique claims entered D2 without independent verification.

5.2 Prompt Utility

The team used four keeper prompts: K1 evidence-ranked actor goals, K2 state-transition/invariant audit, K3 hostile boundary miner, and K4 contradiction adjudicator. K1 earned its keep by forcing exactly 20 ranked, cited goals; K2 exposed paid overflow, unawaited persistence, and receipt/checkout separation; K3 produced the best nontrivial test boundaries; and K4 most directly caught stale prose and CI claims. The generic first-contact starter prompt did not earn a place in the final evidence beyond orientation, and the fragility prompt produced prioritization hypotheses rather than proof of failure. Local K1, K3, and K4 outputs were not useful because they lacked repository grounding; local K2’s explicit N/A answers were useful as an honesty check.

5.3 Model-Specific Strengths and Weaknesses

- **Codex Sol** connected Flutter UI, state, Firestore, routes, workflow, and history into testable invariants; first-pass summaries could still over-trust domain labels or promote system behavior into user goals.
- **GPT-5.6-terra** most consistently distinguished evidence from inference and found silent failures; its wording still inherited *eligibility* and *transaction*, and its OpenAI/Codex lineage limits independence from Sol.
- **Gemini** quickly produced a broad, independent inventory of visible behavior; it regularly promoted UI labels and README prose into stronger guarantees than the code supports.
- **Local Llama 3 8B** followed fixed output formats reasonably well; with only manually pasted README and `app_state.dart` context, it substituted plausible guesses and fabricated citations for repository evidence.

5.4 Local Model

All four keeper prompts were attempted with Llama 3 8B through Ollama. The local-model session could not browse the repository, so only README and `app_state.dart` were pasted. K2 honestly marked two missing-context flows N/A; K1 fabricated citations, K3 supplied 25 generic uncited boundaries, and K4 reasoned from dependencies rather than production code. The local output was retained as comparison evidence, but was not trusted without source verification.

6 Demo Video

The demonstration shows WolfBite running and the Project 1a test suite executing, including both passing and failing results. It is available at WolfBite Project 1a Demo Video.

7 Conclusion

Across 86 team-designed tests, 82 passed, three exposed user-facing mismatches, and one exposed a test-scope mismatch. Coverage

reached 77.5% of executable lines and 18 of 20 main use-case outcomes, but exception and persistence paths remained weakest. Cross-model checking prevented unsupported README and domain claims from becoming requirements and sharpened our assessment of the inherited suite's blind spots.

Table 15: Complete active-test results, UC1–UC3 (15 tests)

Test	Why we tried it	Expected	What happened
test_uc01_valid_registration_creates_account_and_profile	UC1 main path requires both an authentication account and a saved profile.	Create the account/profile, end the initial session, and open login.	PASS: exact credentials and UID were used; profile save, sign-out, and /login navigation occurred.
test_uc01_blank_fields_prevent_submission	UC1 extension 2a rejects missing required information.	Show field errors without authentication or profile writes.	PASS: all four validation messages appeared and neither service was called.
test_uc01_invalid_email_prevents_submission	UC1 extension 2a rejects invalid account information.	Show an email error and make no account request.	PASS: "Enter a valid email" appeared and no authentication/profile request occurred.
test_uc01_rejected_registration_displays_error	UC1 extension 3a says rejected creation must be reported.	Show the error, save no profile, and remain on signup.	PASS: "Email already in use" appeared; no save, sign-out, or navigation occurred.
test_uc01_profile_save_failure_does_not_complete_registration_flow	UC1 extension 4a separates account creation from profile persistence.	Report the failed profile save and do not claim completion.	FAIL: the Firestore exception escaped with no message; sign-out and navigation did not occur.
test_uc02_valid_credentials_authenticate_and_open_shopping_features	UC2 main path requires valid credentials to provide protected access.	Authenticate once and navigate to the shopping interface.	PASS: exact credentials were sent once and /scan opened.
test_uc02_invalid_email_prevents_authentication	UC2 extension 2a rejects an invalid email locally.	Show an email error and make no authentication request.	PASS: "Invalid email" appeared; authentication and protected navigation did not occur.
test_uc02_short_password_prevents_authentication	UC2 extension 2b defines the six-character boundary.	Reject a five-character password without authentication.	PASS: "Min 6 chars" appeared and no authentication request occurred.
test_uc02_rejected_credentials_leave_shopper_signed_out	UC2 extension 3a requires rejected credentials to preserve signed-out state.	Report rejection and withhold protected navigation.	PASS: "Invalid credentials" appeared, login remained visible, and /scan did not open.
test_uc02_second_submission_is_blocked_while_sign_in_is_pending	UC2 extension 3b prevents duplicate pending submissions.	A second action creates no second Firebase request.	PASS: progress replaced the active button and Firebase received exactly one request.
test_uc03_logout_action_requests_authentication_sign_out	UC3 steps 1–2 require the logout action to end authentication.	Call signOut exactly once.	PASS: the logout control called signOut once.
test_uc03_successful_sign_out_returns_shopper_to_login	UC3 step 4 makes sign-in available after logout.	Remove protected access and navigate to login.	PASS: successful sign-out requested /login.
test_uc03_successful_sign_out_removes_account_specific_local_state	UC3 step 3 requires account-specific local state cleanup.	Clear basket, balances, and loaded account state.	PASS: signed-out state cleared basket/balances and reset the loaded flag.
test_uc03_sign_out_failure_leaves_shopper_signed_in	UC3 extension 2a requires failed sign-out to remain failed safely.	Preserve the session and handle the authentication error.	FAIL: login navigation was withheld, but the exception escaped without explanation.
test_uc03_existing_signed_out_state_still_returns_to_login	UC3 extension 4a covers an already-signed-out state.	Present login and hide protected shopping features.	PASS: login appeared and the protected Scan Product interface was absent.

Table 16: Complete active-test results, UC4–UC6 (15 tests)

Test	Why we tried it	Expected	What happened
test_uc04_current_month_saved_session_restores_basket_and_balances	UC4 main path restores the shopper's saved current-month state.	Load basket, quantities, product data, and benefits.	PASS: MILK usage, product identity, quantity, name, and nutrition were restored.
test_uc04_missing_saved_session_starts_empty	UC4 extension 2a covers a shopper with no saved state.	Finish loading with an empty session.	PASS: basket/balances were empty and an empty Firestore scaffold was saved.
test_uc04_partial_saved_session_uses_empty_values_for_missing_sections	UC4 extension 3a permits partial persisted state.	Restore available data and default missing collections to empty.	PASS: MILK balance restored and the missing basket became empty without failure.
test_uc04_missing_nutrition_uses_zero_values	UC4 extension 3b defines missing-nutrition defaults.	Restore the product with zero-valued nutrition.	PASS: calories, fat, sodium, sugar, and protein all became 0.0.
test_uc04_previous_month_session_resets_basket_and_used_benefits	UC4 extension 4a covers benefit-period rollover.	Clear the old basket and reset used benefits.	PASS: a prior-year session produced an empty basket and zero MILK/CEREAL usage.
test_uc05_matching_upc_displays_product_information	UC5 main path identifies a matching UPC without basket mutation.	Show product name, category, code, and nutrition.	PASS: exact UPC was queried, details appeared, and the basket stayed unchanged.
test_uc05_empty_manual_entry_performs_no_lookup	UC5 extension 1b defines blank manual entry.	Make no lookup and leave product/basket state unchanged.	PASS: the service was not called, no result appeared, and basket stayed empty.
test_uc05_unknown_upc_reports_not_found_and_clears_result	UC5 extension 2b prevents a stale result after no match.	Report not found, clear the prior product, and preserve basket state.	PASS: the message appeared, prior result cleared, and basket remained empty.
test_uc05_lookup_failure_displays_error	UC5 extension 2c requires lookup failure to be visible.	Show an error with no product or basket mutation.	PASS: service failure became a visible error and no product appeared.
test_uc05_second_lookup_is_ignored_while_first_is_pending	UC5 extension 2a prevents concurrent lookup duplication.	A repeated action creates no second service call.	PASS: the pending first request blocked a second call.
UC6-T1identifiedproduct->normalizednutrition	UC6 requires raw APL nutrients to feed the product review.	Normalize all seven displayed nutrition/eligibility fields.	PASS: all seven normalized fields matched.
UC6-T2missingornonnumericnutrients->zerodefualts	UC6 extension 1a covers malformed or absent nutrients.	Substitute numeric zero values without crashing.	PASS: calories, fat, sodium, and protein defaulted to 0.0.
UC6-T3exactthresholds->allinclusivequalitiesapply	UC6 extension 1b checks every equality boundary.	Award all qualities whose inclusive thresholds are met.	PASS: all six expected qualities were present.
UC6-T4nutritionmatchingnorable->noqualities	UC6 extension 2a covers a product outside every rule.	Return an empty qualities list.	PASS: no quality was returned.
UC6-T5morethanthreequalities->compactviewshowsthree	UC6 extension 2b bounds the compact presentation.	Render exactly three quality tooltips.	PASS: exactly three tooltips rendered.

Table 17: Complete active-test results, UC7–UC10 (18 tests)

Test	Why we tried it	Expected	What happened
UC7-T1 eligible same-category candidates -> ranked healthier list	UC7 main path requires eligible, healthier same-category ranking.	Return only the two healthier candidates in score order.	PASS: best then good returned in ascending score order.
UC7-T2 candidate pool -> excludes original, ineligible, and other category	UC7 extensions require three candidate exclusions.	Exclude the original, ineligible, and other-category products.	PASS: no candidate remained.
UC7-T3 more than maximum healthier candidates -> first five ranked	UC7 step 4 limits presentation to five results.	Return the five best scores from seven qualifying products.	PASS: exactly scores 0.1 through 0.5 returned.
UC7-T4 candidate missing nutrition -> zero-score candidate can qualify	UC7 extension 2a records the missing-nutrition scoring risk.	Missing nutrition produces a qualifying zero score.	PASS: UPC missing qualified with score 0.0.
UC7-T5 no better candidate -> empty alternatives	UC7 extension 2c rejects equal-score candidates.	Return no alternatives.	PASS: the alternatives list was empty.
UC7-T6 blank category -> no search results	Probe the blank-category boundary attributed to the UC7 screen flow.	Perform no search and return no alternatives.	SCOPE FAIL: direct service use returned a product; the production screen guards the call.
UC8-T1 identified covered product -> basket and usage increment	UC8 main path couples a covered addition with benefit usage.	Add quantity one and increase MILK usage to one.	PASS: addition, basket quantity, and usage assertions matched.
UC8-T2 exhausted allowance -> unchanged basket and usage	UC8 extension 1a enforces the category cap.	Reject addition without basket or usage mutation.	PASS: the operation returned false and state was unchanged.
UC8-T3 existing UPC -> quantity flow without duplicate line	UC8 extension 2a delegates a duplicate UPC to quantity increase.	Keep one line at quantity two and usage two.	PASS: one quantity-two line remained and usage became two.
UC8-T4 signed-out shopper -> rejected unchanged basket	UC8 extension 2b requires authentication for mutation.	Reject the addition and preserve empty state.	PASS: the operation returned false with no basket/balance mutation.
UC8-T5 raw category -> canonical in-memory basket state	Category spacing/case must not split benefit balances.	Canonicalize both basket and balance keys.	PASS: both locations stored MILKPRODUCTS.
UC9-T1 available alternative -> exact product submitted	UC9 main path must submit the chosen alternative, not the original.	Send exact fields to addItem and report success.	PASS: the exact alternative was submitted once and success appeared.
UC9-T2 rejected alternative -> must not report success	UC9 extensions require the UI to honor state rejection.	Suppress the success message after a false return.	FAIL: UI displayed "Added healthier item: Better cereal."
UC10-T1 ready -> source dialog cancelled -> ready	UC10 extension 1a covers cancellation before source selection.	Preserve the ready prompt and selection control.	PASS: both ready-state elements remained visible.
UC10-T2 source chosen -> image selection cancelled -> ready	UC10 extension 1a also covers platform-picker cancellation.	Return to a retryable ready state without processing.	PASS: selection and upload/analyze controls remained available.
UC10-T3 OCR failure -> visible error and ready state	UC10 extension 2a requires OCR failure to be explained.	Show the API error and permit retry.	PASS: API 500 text and the selection control were visible.
UC10-T4 OCR text without candidate -> zero-candidate status	UC10 extension 3a covers OCR text without a 12–14 digit candidate.	Report no candidates and permit retry.	PASS: the zero-candidate status and selection control appeared.
UC10-T5 spaced or hyphenated code -> no candidate	UC10 extension 3b defines separated-digit parsing behavior.	Treat spaced/hyphenated groups as no candidate.	PASS: the screen reported no UPC candidates.

Table 18: Complete active-test results, UC11–UC15 (14 tests)

Test	Why we tried it	Expected	What happened
test_uc11_multiple_recognized_products_are_each_added	UC11 main path evaluates and adds each distinct receipt product.	Create two lines and increment both categories.	PASS: both products and both used counts increased by one.
test_uc11_duplicate_recognized_upc_increments_but_addition_count_is_undercounted	UC11 extension 2a predicts quantity/report divergence for a duplicate.	Increment one line to two while the second call returns false.	PASS: one quantity-two line remained; returns were true then false.
test_uc11_product_without_nutrition_uses_zero_defaults	UC11 extension 1a defines missing-nutrition defaults.	Add the product with zero-valued nutrition.	PASS: product was added and calories/protein defaulted to 0.0.
test_uc12_basket_displays_items_quantities_and_categories	UC12 main path presents current products and quantities.	Show both names/categories and a total-items summary.	PASS: both product/category pairs and "Total Items" appeared.
test_uc12_empty_basket_shows_empty_state	UC12 extension 1a requires an explanatory empty state.	Show an empty message and a way to start scanning.	PASS: "Your basket is empty" and "Start Scanning" appeared.
test_uc12_covered_and_paid_lines_for_same_upc_display_separately	UC12 extension 1c distinguishes covered and paid quantities.	Render the same product separately under MILK and PAID.	PASS: "Whole Milk" appeared twice under the two categories.
test_uc13_expanding_item_shows_nutrition_facts	UC13 main path exposes stored nutrition on request.	Reveal the stored calorie, fat, sodium, sugar, and protein values.	PASS: the panel showed the exact stored calorie, fat, and sodium values.
test_uc13_missing_nutrition_map_shows_zero_defaults	UC13 extension 2a covers a completely absent nutrition map.	Show zero defaults without a crash.	PASS: the panel showed 0.0cal and 0.0mg.
test_uc13_collapsing_hides_nutrition_facts	UC13 step 4 returns from details to the basket summary.	Hide the panel and restore the collapsed control.	PASS: facts disappeared and "Show Nutritional Info" returned.
test_uc14_increase_button_calls_increment_when_under_cap	UC14 main path delegates an under-cap increase to state.	Call incrementItem for the correct product/category once.	PASS: incrementItem('1', 'MILK') was called once.
test_uc14_increase_button_still_active_at_cap_and_reports_paid_intent	UC14 extension 1a continues at-cap requests into UC15.	Keep the control active, signal paid intent, and delegate once.	PASS: tooltip said "Will add as paid" and state was called once.
test_uc15_increment_at_cap_creates_separate_paid_line	UC15 main path creates shopper-paid quantity after coverage is exhausted.	Create one PAID line without increasing MILK usage.	PASS: a quantity-one PAID line appeared and MILK usage stayed three.
test_uc15_second_paid_request_increments_existing_paid_line	UC15 extension 2a prevents duplicate paid lines.	Keep one PAID line at quantity two after two requests.	PASS: exactly one PAID line existed with quantity two.
test_uc15_paid_category_has_no_allowance_cap_confirming_unlimited_display	UC15 extension 4a explains why PAID appears unlimited.	Store a null allowance for PAID.	PASS: balances['PAID']['allowed'] was null.

Table 19: Complete active-test results, UC16–UC20 (13 tests)

Test	Why we tried it	Expected	What happened
test_uc16_decrease_covered_quantity_updates_quantity_and_usage	UC16 main path couples covered decrement with benefit release.	Decrease quantity and covered usage by one.	PASS: both quantity and usage decreased correctly.
test_uc16_quantity_reaching_zero_removes_selection	UC16 step 4 removes a zero-quantity selection.	Remove the product and reduce usage to zero.	PASS: product was removed and usage became zero.
test_uc16_paid_quantity_is_decreased_before_covered_quantity	UC16 extension 1a defines paid-first removal.	Decrease PAID first and preserve covered quantity/usage.	PASS: PAID was removed first and covered state stayed unchanged.
test_uc16_missing_product_causes_no_quantity_change	UC16 extension 1c covers an unknown matching product.	Leave basket quantities and benefit usage unchanged.	PASS: no quantity or usage changed.
test_uc16_benefit_usage_does_not_go_below_zero	UC16 extension 3a requires a nonnegative usage clamp.	Keep usage at zero after decrement.	PASS: usage remained zero.
test_uc17_clear_removes_all_products_and_reverses_usage	UC17 main path clears products and provisional usage.	Empty the basket and reverse associated usage.	PASS: basket cleared and usage was reversed.
test_uc17_clear_reverses_usage_across_multiple_categories	UC17 main path must handle all affected categories.	Remove every product and reverse each category's usage.	PASS: all products and category usage were reversed.
test_uc17_missing_category_balance_still_removes_product	UC17 extension 4a covers absent category balance data.	Remove the product without requiring a balance update.	PASS: product was removed and missing balance data caused no issue.
test_uc17_clear_does_not_make_usage_negative	UC17 extension 4b clamps usage below the removed quantity.	Clear the basket and leave usage at zero.	PASS: usage remained zero rather than negative.
test_uc18_paid_category_is_displayed_as_unlimited	UC18 extension 2a records the PAID presentation quirk.	Show "Unlimited" and the PAID used-item count.	PASS: PAID displayed as unlimited with correct usage text.
test_uc18_usage_at_allowance_shows_full_progress	UC18 extension 3a checks the exact-cap boundary.	Show full allowance text and progress 1.0.	PASS: usage text was correct and progress was full.
test_uc19_qr_code_contains_current_basket_data	UC19 main path requires a scannable checkout handoff.	Render a QR widget for the nonempty basket.	PASS: a QRImageview rendered; payload content was not asserted.
test_uc20_checkout_clears_basket_but_retains_benefit_usage	UC20 main path distinguishes purchased usage from provisional basket state.	Empty the basket while retaining recorded benefit usage.	PASS: basket cleared and benefit usage remained unchanged.

Table 20: Complete active-test results, post-curation gap tests (11 tests)

Test	Why we tried it	Expected	What happened
UC8 accepted addition persists basket and benefit usage	UC8's original tests proved local mutation but not persistence.	Save the milk line and one used MILK benefit.	PASS: Firestore contained UPC milk-1 and MILK usage one.
available allowance increments quantity and covered usage	UC14 needed the real under-cap transition, not only UI delegation.	Increase covered quantity/usage from one to two with no PAID line.	PASS: both values became two and no PAID line appeared.
missing covered product leaves quantity and usage unchanged	UC14 extension 2a prevents an unknown UPC from mutating another line.	Preserve existing quantity and MILK usage at one.	PASS: basket quantity and benefit usage were unchanged.
UC14 quantity increase persists quantity and covered usage	UC14 local mutation does not prove the resumed-session state.	Persist basket quantity two and MILK usage two.	PASS: both persisted values were two.
missing original product cannot create a shopper-paid line	UC15 extension 2b prevents paid quantity without a source product.	Keep one original line, no PAID line, and usage one.	PASS: basket length and usage stayed one; no PAID line appeared.
UC16 quantity decrease persists quantity and released usage	UC16 needed durable evidence for both changed values.	Persist basket quantity one and MILK usage one after decrement.	PASS: both persisted values were one.
UC17 clear persists empty basket and reversed usage	UC17 needed proof that clearing and usage release are durable.	Persist an empty basket and zero MILK usage.	PASS: persisted basket was empty and MILK usage was zero.
cancelling clear leaves the basket unchanged	UC17 extension 2a requires cancellation before destructive state change.	Never call clearBasket; keep Milk visible.	PASS: no clear call occurred and the Milk line remained.
confirming clear invokes the state transition once	UC17 confirmation must delegate exactly once.	Selecting Clear All calls clearBasket once.	PASS: clearBasket was invoked exactly once.
checkout saves an empty basket and retains used benefits	UC20 required persistence against real production checkout behavior.	Save an empty basket while retaining MILK usage two.	PASS: persisted basket was empty and MILK usage remained two.
save failure leaves the local basket cleared and reports failure	UC20 extension 4a checks the local-clear-before-save failure order.	Throw a Firebase error while leaving basket empty and usage one.	PASS: exception propagated, basket stayed empty, and usage stayed one.